One Agent or a Team? An Introduction to Orchestration

ME/CE/AM 295

Agentic AI in Scientific Discovery
9 units (3-5-1)

Instructor: Chiara Daraio
TAs: Xiaoxiao (Alice) Xiong
Hyoseo (Lauren) Yoon

California Institute of Technology
Division of Engineering and Applied Science

Lecture 9
Wednesday 28 October 2026

WHERE WE LEFT OFF · MONDAY (LECTURE 8)

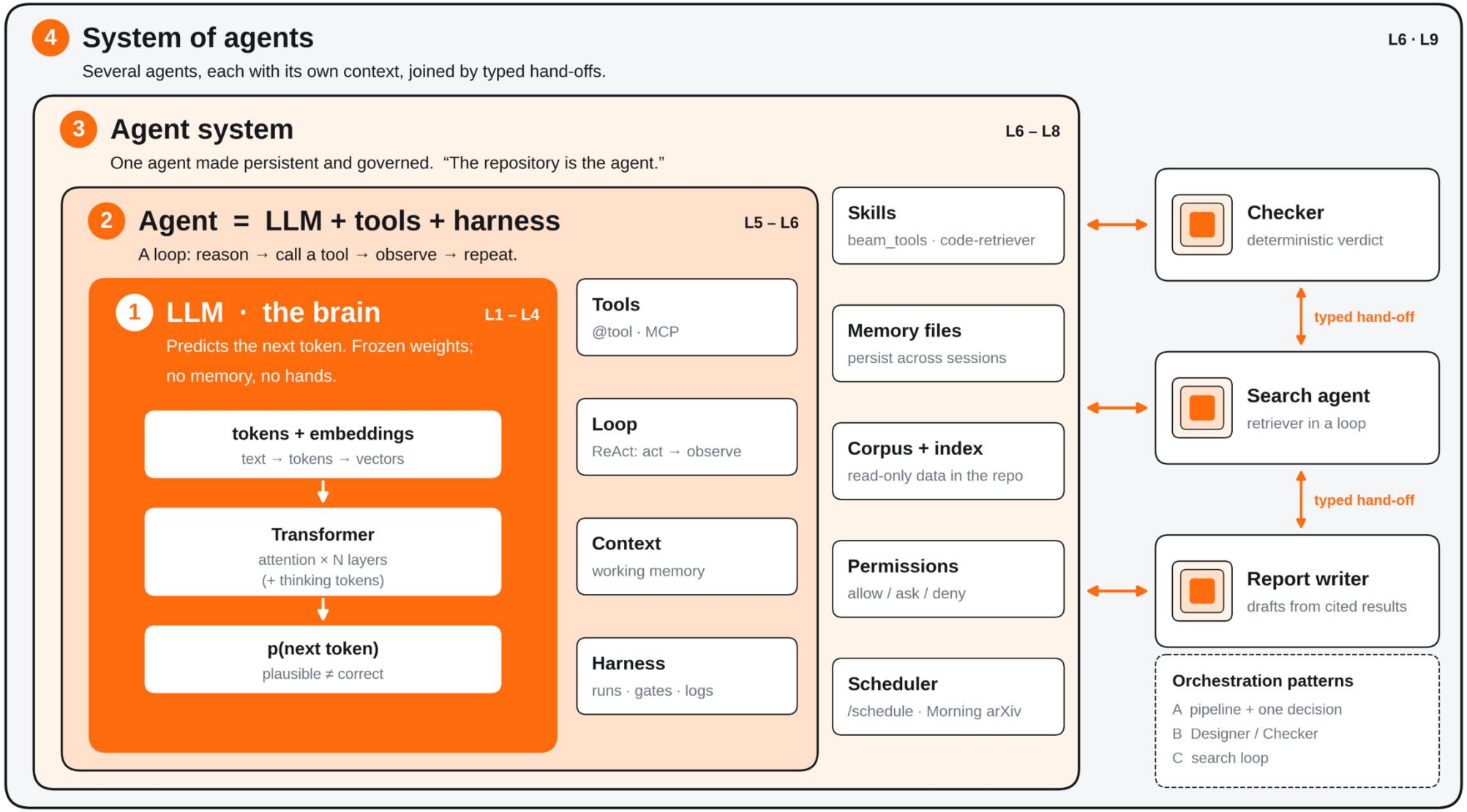
Your agent can now read: a corpus, a retriever, a citation check — skill #3. On Monday you drew your own agent as boxes. Today: when a box earns a second agent, and how the boxes talk.



WORKSHOP	PART 1	PART 2	PART 3	PART 4
Your worksheets	What is known	One agent or many?	Design the structure	Tools

0:00 worksheets → 0:11 literature → 0:23 one vs many → 0:29 in the news → 0:31 structure → 0:43 tools → 0:52 close

The Class, So Far



The Four Rules on a Messy Project — a Filled-in Worksheet

An imaginary project, chosen because it has none of the beam's conveniences. The same boxes (1–7) you fill in next.

THE PROJECT A biologist records rattlesnake rattles in the field and wants to classify each recording by the temperature it was made at (snakes rattle faster when warm). 600 clips. The temperature labels came from the site logbooks and some are wrong. There is no formula that says whether a label is right.

R = run it: a fixed input gives a fixed output → a tool. **D** = look and decide: you read an output and choose what happens next → an agent candidate. (Rule 1)

1 Workflow by hand

1–3 load, segment, features — R
4 doubtful label: keep / relabel / discard — D
5 train the classifier — R
6 read the confusion matrix — D
7 report accuracy + CI — R

2 The one decision

Step 4. A clip whose label disagrees with its rattle rate: keep, relabel (with a reason), or discard. Stop when every flagged clip is resolved. Escalate to the PI if a whole site looks wrong.

3 Its check — not the LLM

60 clips labelled independently by two people (κ > 0.8): relabel precision and recall against them. Plus a physical bound: the rattle rate must fit the published range for that temperature. Both are scripts.

4 Its baseline — no agent

A fixed rule on the same features: noisy clip → discard; rate outside the bound → relabel; else keep. Same held-out clips, five repeats. The agent must beat it — or the rule suffices, and that is the result.

5 Tool list (in → out, units)

segment_rattles(wav) → intervals [s]
features(interval) → rate [Hz], SNR [dB]
rate_bound(T [° C]) → [Hz] range
train_eval(split) → accuracy, 95 % CI
All exist? Y — steps 1–3, 5, 7.

6 A second agent? Why?

No. Step 6 is also a D step, but it needs the same information, the same permissions and the same judgment as step 4 — none of Rule 3's three reasons. It stays with the human for now.

7 The redeploy test

CLAUDE.md — spec, may-not list
skills/ — four tools · settings.json — no network
memory/ by PR · LOG.md · lock file · model pinned
make smoke — from a fresh clone

2 D STEPS OF 7.
1 AGENT, 4 TOOLS.
A CHECK THAT IS NOT THE MODEL.
A BASELINE TO BEAT.

Your Worksheets — What the Four Piles Say

4 PILE 1

Decision + check + baseline

Ready. Boxes 2–4 are the core of Sunday's proposal — write them up.

6 PILE 2

A decision, no pass/fail check

Rule 2: finish "I know this box was right because ___". The blank cannot be the model.

1 PILE 3

No decision yet

Rule 1: write the workflow by hand and mark R / D. One D step you can check is enough.

1 PILE 4

The decision is literature search

Is the literature the research loop, or support for it? Usually support — a tool (Lab 5), not the agent.

TA clinic: today's office hour, right after class. Bring the redrawn diagram; leave with a yes or no on the proposal bar and one next step.

Counts (orange) from the Monday-night sort of the sheets handed in.

Rule 1 — Start From What You Cannot Script

Start from the workflow you already run by hand, and mark where you stop and think.

THE PROCEDURE

1. **Write the workflow as you actually do it:** raw input to the number you report, one step per line.
2. **Mark each step R or D.** R = run it: a fixed input gives a fixed output (load a file, run the solver, compute RMSE). D = look and decide: you read an output and choose what happens next.
3. **Every R step becomes a tool.** Every D step is an agent candidate.
4. **Pick the one D step** whose outcome you can check and whose baseline you can run. That is your agent. Everything else is a tool it calls.

D STEPS FROM YOUR LAB REPORT 1s

- Naming the fault behind a failed check and choosing a remedy, or declining to report
- Deciding whether a counterexample calls for a gain change or a new controller structure
- Choosing where in a five-dimensional space the next simulation campaign should go
- Judging which of 300 abstracts deserve a full read, and when to stop searching
- Deciding that the spec itself is wrong and asking a person

If your project is truly a pipeline, consider building an agent that operates, monitors and audits that pipeline, or individual steps within it

Worksheet boxes 1 and 2. The beam agent of Lecture 6 passes this test: lookup and deflection are R; "which section next, given this failure" is D.

Rule 2 — Cut at Verification Boundaries, Not Task Boundaries

"How big should each piece be?" is the wrong question.
A piece is the right size when something that is not the LLM can check its output.

1

The piece has a check that is not the LLM

A formula (deflection, equilibrium, residual vs noise floor), a unit test, a held-out measurement, a human label. Keep it. The check's threshold comes from the spec or the math, never from the model.

2

The piece has no check

Split it until each part has one — or it is not an agent decision at all: make it a tool. Pieces that cannot be checked individually are where errors compound silently, and nothing raises.

3

The only possible check is another LLM

Allowed where no formula exists ("is this summary accurate?"), but you have bought a second faithfulness problem. Give the critic different information from the worker, and put a deterministic or external check at the end.

4

The test, in one sentence

Point at each box and finish: "I know this box was right because ___." If the blank is "the model said so", the box is not done.

Divide agents per decision and verification, not per task/role

Rule 3 — One Agent per Context, Not per Task

"How specialized should each agent be?"
Split into a second agent only for one of three reasons; otherwise it is a pipeline stage wearing a hat.

SPLIT · INFORMATION

The two jobs would mislead each other if one agent held both. The Checker must not see the Designer's reasoning, or it rationalizes it. Demo B's Checker got files, not the Researcher's transcript.

SPLIT · PERMISSIONS

The thing that launches cluster jobs or moves a robot arm should not be the thing that edits the spec. Different allow-lists in settings.json mean different agents.

SPLIT · JUDGMENT

You need a second opinion on the same object: an adversary hunting for a counterexample, a reviewer with no memory of the project. Same model, different context: independence of context, not of judgment.

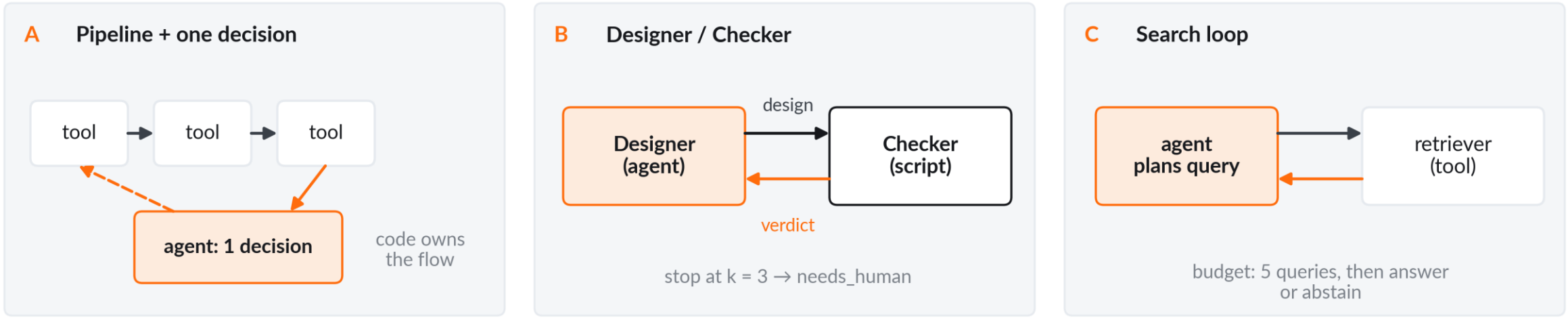
DO NOT SPLIT FOR

Pipeline stages (load, clean, run, tabulate): those are tools. **"It feels cleaner":** that is a cost with no evidence. **Generalist vs specialist:** a generalist with many tools is fine for exploring; a specialist with few tools and a typed contract is what you can evaluate, reproduce and audit in Week 9. Build the specialist.

Worksheet box 6. Most capstone projects need one or two LLM agents and three to five tools. Four agents and zero tools is the ratio backwards.

Your Diagrams Already Contain Orchestration

Across the sheets, three shapes recur. Each one is an answer — usually given by accident — to the question "who decides what happens next?"



A · code runs the steps; the agent makes one call a script can't.

B · Lecture 6: two roles, a typed hand-off, a stop at k = 3.

C · Lecture 8 slide 10: agentic search is a loop around the retriever.

Today's question: when does a box earn a second agent — and when it does, how should the boxes talk?

Shapes redrawn generically from the worksheets; no individual sheet is shown.

Orchestration — Who Acts Next, and With What

The part of the system that decides which agent or tool runs next, what it may see, and what it must hand back. You have built it three times already.

THE ANALOGY · A BUILDING SITE

General contractor orchestrator: plans, assigns, never pours concrete

Subcontractors workers: one trade, their own tools and permits

Submittals hand-offs: a typed document, stamped before it moves

The inspector checker: paid by no subcontractor, reads the drawings, not the gossip

1 · CONTROL FLOW

Who decides the next step — your code, or the model?

2 · SHARED STATE

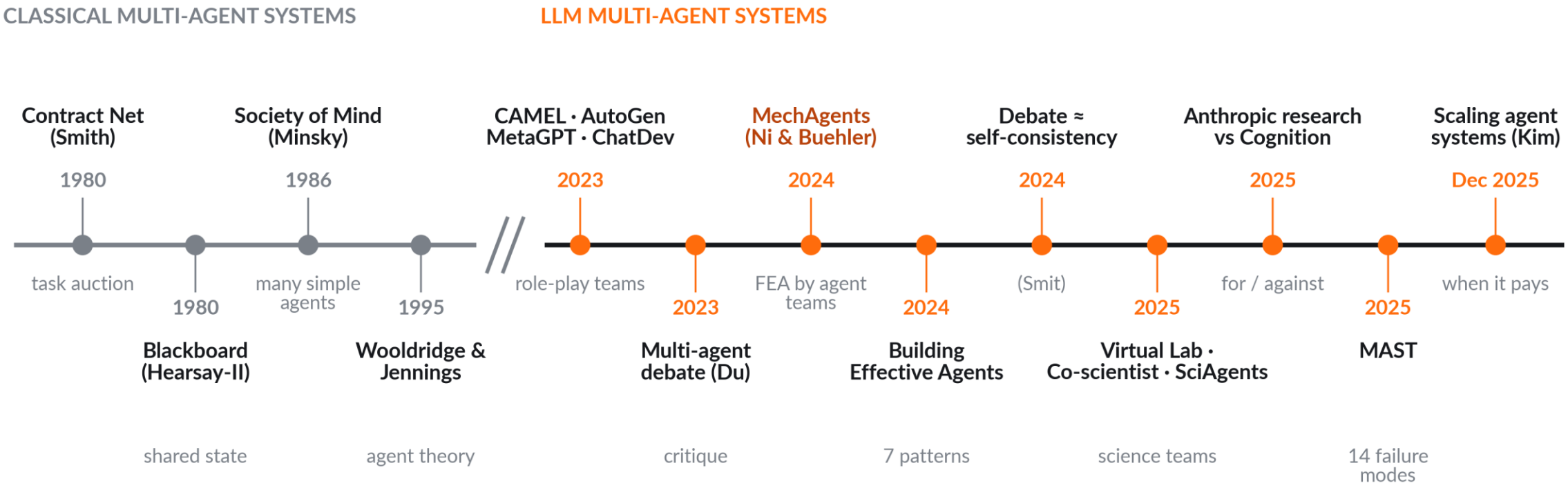
What does each agent see — the whole transcript, a shared file, or only its brief?

3 · HAND-OFF

What crosses between agents — free text, or a typed contract a script can check?

Forty Years of Agents That Talk to Each Other

LLM teams reinvent distributed-AI ideas: task auctions, shared blackboards, societies of simple agents. What is new: each agent is a language model — fluent, and wrong in correlated ways.

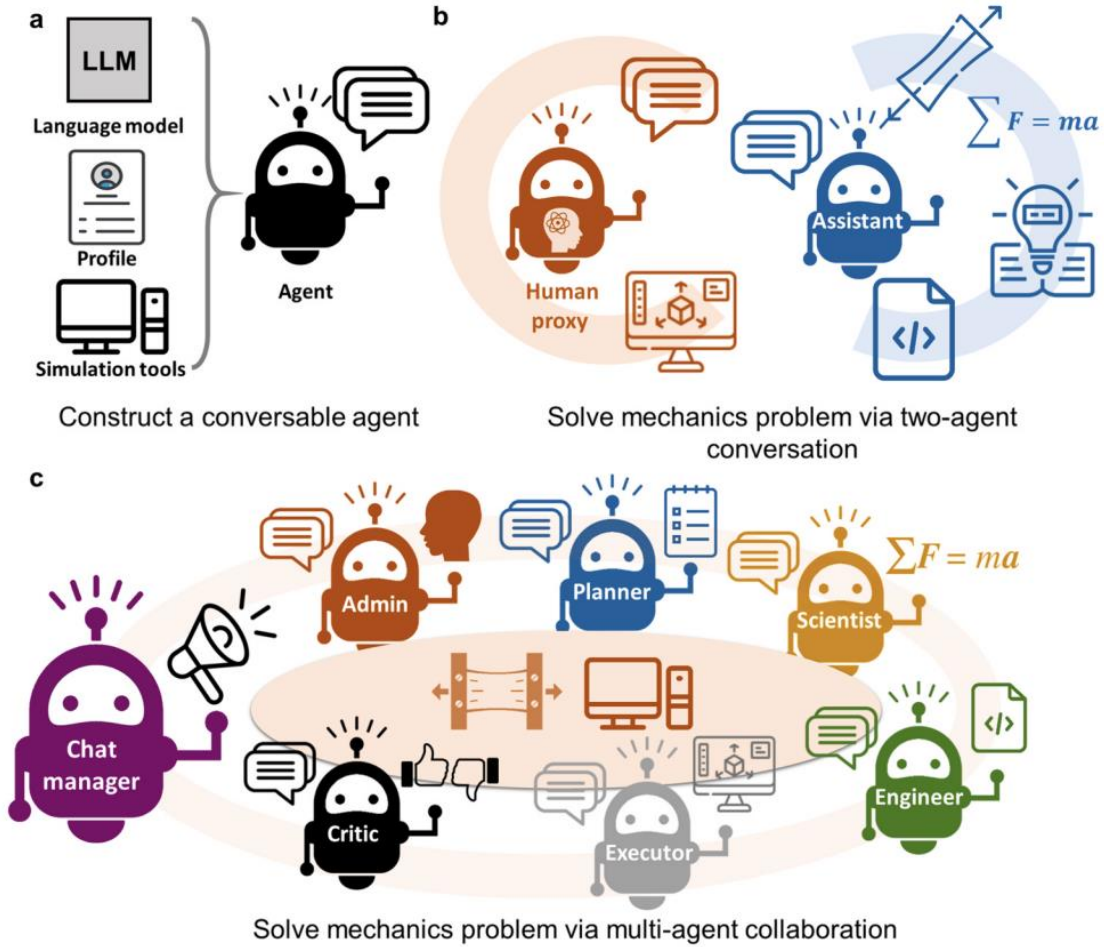


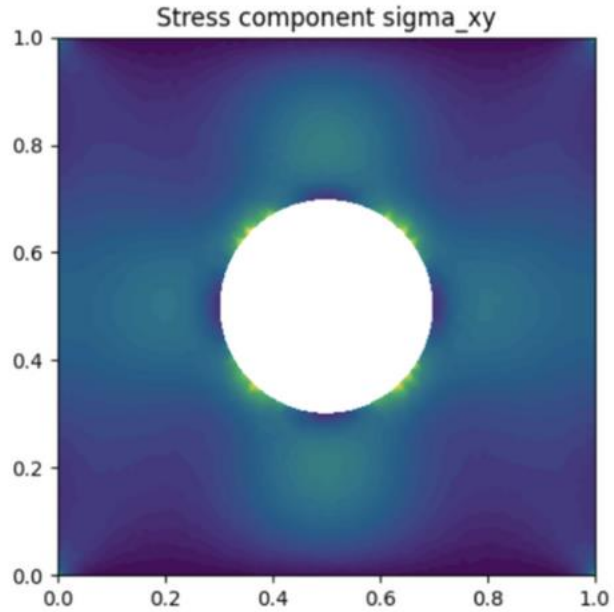
Contract Net = orchestrator–workers · blackboard = shared state · Society of Mind = specialization.

Full citations on the References slide. Classical MAS: Smith 1980; Erman et al. 1980; Minsky 1986; Wooldridge & Jennings 1995. Mechanics: Ni & Buehler 2024 (MechAgents).

MechAgents (2024)

Extreme Mechanics Letters 67 (2024) 102131





The paper's main finding: **organizing the same model into a team with a separate critic caught a physically wrong result that a single self-correcting agent accepted.**

Figures: Ni & Buehler (2024), Extreme Mechanics Letters 67, 102131, Figs. 1 and 2 (arXiv:2311.08166, CC BY -NC-SA 4.0).

Multi-Agent Science — Read the Check Column First

Four published teams. Where the check is an experiment or a script, the claim is strong; where it is another LLM, you have bought a second faithfulness problem.

System	The team	What checks the output
Virtual Lab **Swanson et al. 2025, Nature**	PI agent + specialist agents + a scientific critic; a human sets the agenda	Wet lab: designed nanobodies expressed and tested
AI co-scientist **Gottweis et al. 2025**	Supervisor + generation, reflection, ranking (tournament), evolution, meta-review	LLM tournament to rank; selected hypotheses tested in vitro
SciAgents **Ghafarollahi & Buehler 2024**	Ontologist, scientist, critic over a knowledge graph built from ~1,000 papers	An LLM critic — no experiment in the paper
Design Agents **Elrefaie et al. 2025**	Styling, CAD and aerodynamics agents for car design	A drag surrogate trained on DrivAerNet++ — a model, not CFD

Each row is also a design choice: who orchestrates (a PI agent, a supervisor, a fixed sequence) and what the team shares (a meeting transcript, a graph, files).

The Evidence — Teams Buy Tokens, Not Intelligence

The strongest numbers, from those building the systems and from controlled comparisons.

+90%

Lead + parallel subagents vs one agent, on an internal research eval of breadth-first questions.

Hadfield et al. 2025

~15×

the tokens of a chat (one agent ≈ 4×). Token use alone explained 80% of the variance on BrowseComp.

Hadfield et al. 2025

+81% … −70%

From decomposable financial reasoning to sequential planning — same tools, prompts, compute.

Kim et al. 2025 · 260 configs

14 modes

in 1,600+ traces; most failures are design and inter-agent misalignment, not the model.

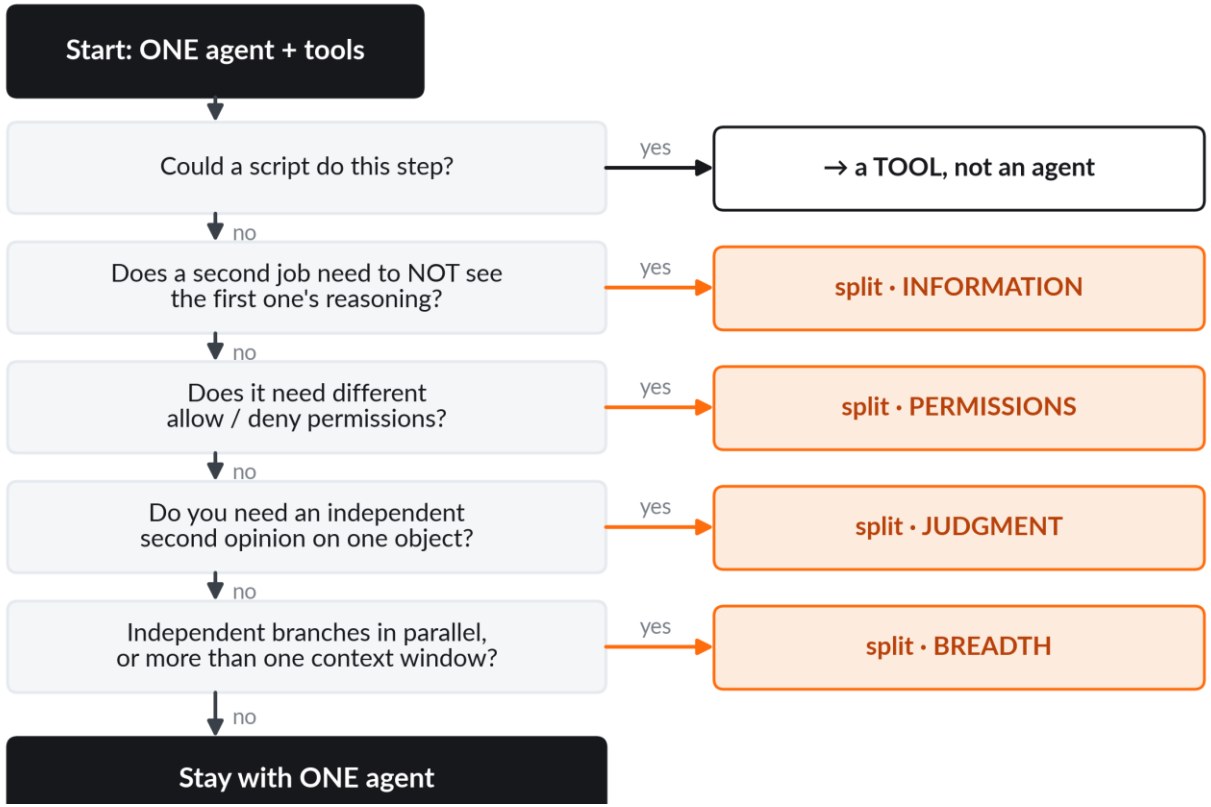
Cemri et al. 2025 · MAST

DISCUSSION POINT 1 If tokens explain 80% of the variance, what single-agent baseline must you run before you can claim your team helps?

Also: multi-agent debate does not reliably beat self-consistency at equal budget (Smit et al. 2024). Anthropic figure: Claude Opus 4 lead with Sonnet 4 subagents vs Opus 4 alone.

One Agent or Several? — The Split Test

Rule 3 (slide 8) gave three reasons to split. The literature adds a fourth. Everything else stays one agent with more tools.



DO NOT SPLIT FOR

Sequential chains where each step needs the last one's full context — this is where teams lose 39–70%.

Pipeline stages (load, run, tabulate): those are tools.

Personas ("you are a materials scientist"): a prompt, not an agent.

TWO LABS, BOTH RIGHT

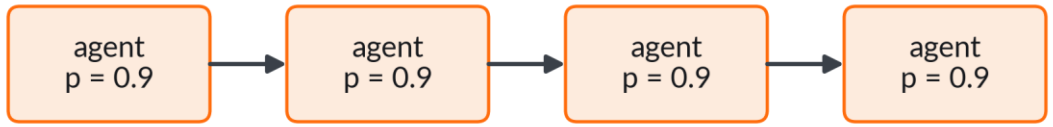
Coding — shared context, many dependencies → one writer (Yan 2025). Research — independent leads → parallel readers (Hadfield 2025).

BREADTH = independent branches, or more material than one context window holds. INFORMATION, PERMISSIONS, JUDGMENT: Rule 3, slide 8.

Why Teams Fail Quietly — Series Systems and the Inspector

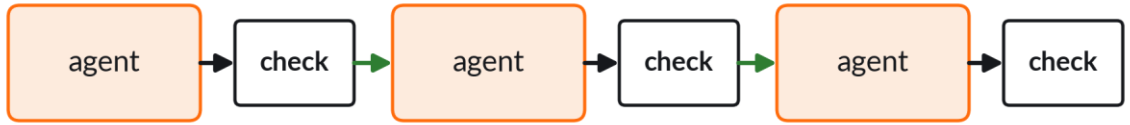
A chain of agents is a series system: it is as reliable as the product of its links. The fix structural engineers use is the one that works here — inspect at the joints.

SERIES — every hand-off unchecked



$0.9^4 \approx 0.66$ — the rule of three, extended

CENTRALIZED — a check at each hand-off



Independent agents: errors ×17.2

Centralized, with validation: ×4.4

Kim et al. 2025 (arXiv v1)

THE COST LINE GROWS TOO

Lecture 4: Cost = N_calls · N_samples × [p_in·n_in + p_out·(n_answer + n_reasoning)]. Every agent adds calls and re-reads context: ≈ 15× a chat.

Beam example: Designer → Reviewer → Reporter, three LLMs, no script: $0.9^3 \approx 0.73$. Designer → script Checker: the joint is inspected.

DISCUSSION POINT 2 Worksheet box 6: which of the four reasons does your second agent meet? If none, cross it out.

×17.2 vs ×4.4 error amplification: Kim et al. 2025, arXiv v1; v3 (Apr 2026) restates it qualitatively — architectures without centralized verification propagate more errors.

In the News This Month — Opposite Moves, Same Week

October 2026: Meta split one agent into fourteen; Anthropic advised folding subagents back into one loop. Both are right, for the same reason.

ONE → FOURTEEN · META 19 Oct

- One agent that knew everything hallucinated in production: right knowledge, applied to the wrong subsystem.
- Fix: a 1,300-line router that does zero domain work, plus 14 specialists, each with isolated context.
- 6 of 7 steps are deterministic code; only the delegated step uses the LLM.
- Reported: column prep 3–4 days → under an hour; about 2/3 of changes land without major edits.

MANY → ONE · ANTHROPIC 22 Oct

- Enterprise agents built as a copy of the org chart: one subagent per team.
- Symptoms: subagent sprawl, a "Frankenprompt", lossy hand-offs, added latency.
- Fix: one loop that loads skills and tools only when needed; one customer ran about 2× faster.
- Keep a subagent for context bloat, parallel work, or tool-selection confusion.

Both split by context, not by org chart. Meta's specialists need knowledge that confuses the others (INFORMATION); Anthropic's shared context and only paid hand-off costs.

Engineering at Meta, "The Thin Orchestrator" (19 Oct 2026) · Anthropic, "When to fold subagents into your main agent loop" + single-agent-loop-kit (22 Oct 2026). Company-reported figures; neither gives a sample size.

Three Questions, Answered on Purpose

For each question, options run from most control in your code (left) to most control in the model (right). The course default is the column you start from.

	Your code ←		→ The model	Course default
Control flow	fixed sequence	graph with conditional edges	orchestrator LLM delegates; peers hand off	graph; an LLM orchestrator only for BREADTH
Shared state	private context + a brief	shared files (a blackboard)	one shared transcript	files in the repo, one writer each; pass paths, not copies
Hand-off	typed schema + script check	typed schema	free text	typed and checked — a stamped submittal

INSTRUCTION HIERARCHY

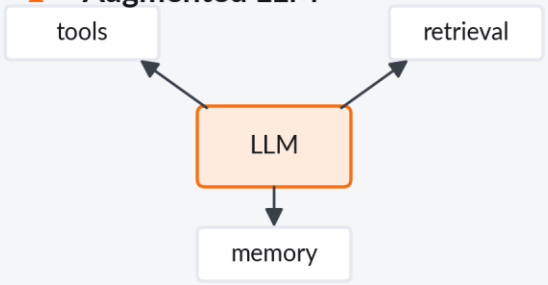
Spec and harness > orchestrator's brief > a worker's output > retrieved text. A worker's output is data, never an instruction (Wallace et al. 2024) — Lecture 7's poisoning lesson, one level up.

Hand-off row is ordered by control: a script-checked schema leaves the model least room. Passing references to files instead of copies: Hadfield et al. 2025, appendix.

Seven Patterns — 1 Building Block, 5 Workflows, 1 Agent

Building Effective Agents names the shapes. You have built four of them; orchestrator–workers is the new one today.

1 Augmented LLM



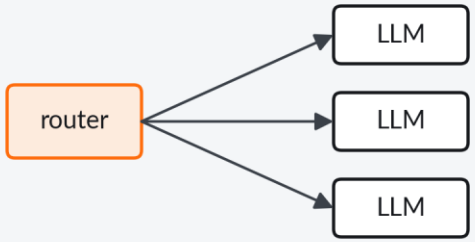
building block · L2–L3

2 Prompt chaining

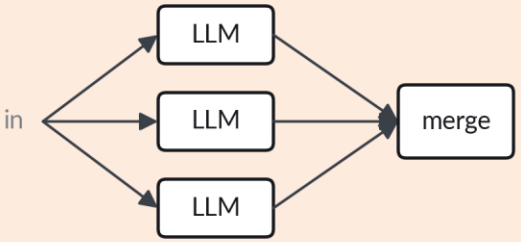


L2 pipeline

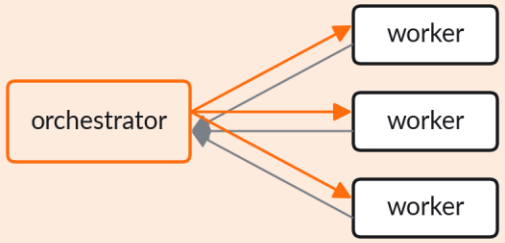
3 Routing



4 Parallelization



5 Orchestrator–workers



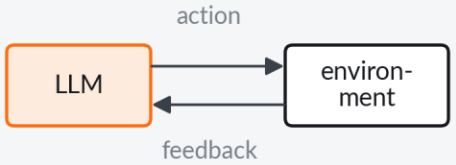
today · new

6 Evaluator–optimizer



L6 Designer/Checker

7 Autonomous agent



L5 loop · L8 search

WORKFLOWS (2–6)

code decides the path

AGENT (7)

the model decides the path

Schluntz & Zhang 2024

Multi-agent topologies map onto these: supervisor = 5 · hierarchical = 5 nested · debate = 4 + 6 · peer hand-off ("swarm", Kim et al.'s decentralized) = 7 with several LLMs.

MAST — Fourteen Ways a Team Fails, in Three Families

MAST = Multi-Agent System Failure Taxonomy. Cemri et al. annotated 1,600+ traces from seven frameworks. The families tell you where to put the fix.

SPECIFICATION / SYSTEM DESIGN · 5

disobeys task or role spec · repeats steps · loses history · doesn't know when to stop

Fix: a written spec per agent; step budget; stop condition in the harness

INTER-AGENT MISALIGNMENT · 6

conversation reset · no clarifying question · derails · withholds · ignores input · reasoning ≠ action

Fix: typed brief with required fields; artifacts in files; "ask" as an allowed output

TASK VERIFICATION · 3

stops too early · no or incomplete verification · incorrect verification

Fix: a deterministic checker outside the team; threshold from the spec

```
[designer] proposes W10x22 for the 20 ft span
[checker]  "W10x22 looks reasonable. PASS."
           (no tool call in the trace)
```

DISCUSSION POINT 3 Which MAST mode is this — and does it matter that W10×22 is in fact the right answer?

Full list of the 14 modes: Appendix A. Mode names follow arXiv:2503.13657 v3 (NeurIPS 2025).

The Orchestration Contract, in Code

An orchestrator–workers loop in the raw SDK. Every rule on the next slide is a line here; none of them is a sentence in a prompt.

```
class Brief(BaseModel):                # the submittal
    objective: str
    output_schema: str                 # e.g. "Finding"
    allowed_tools: list[str]
    boundaries: str                    # what NOT to do
    max_tool_calls: int = 10

class Plan(BaseModel):
    briefs: list[Brief]                # 1-4; more needs a reason

for rnd in range(MAX_ROUNDS):          # MAX_ROUNDS = 3
    plan = client.messages.parse(model=MODEL, max_tokens=4000,
        system=ORCH_SPEC, messages=history,
        output_format=Plan).parsed_output
    found = [run_worker(b) for b in plan.briefs]  # own context
    verdict = check(found)             # a script, not a model
    spent += sum(f.tokens for f in found)
    if verdict.ok or spent > BUDGET:
        break
    history.append(feedback(verdict))
if not verdict.ok:
    escalate("needs_human", found)     # Lecture 6: k = 3
```

`Brief`
the hand-off is a type: objective, schema, tools, boundaries, budget

`run_worker`
a fresh context per brief — the worker never sees the plan's reasoning

`check`
deterministic; the orchestrator cannot talk it round

`MAX_ROUNDS · BUDGET`
stop conditions live in code, not in the prompt

`needs_human`
the honest exit: a person decides

MODEL = the course model ID (pinned in the repo). run_worker() is the Lecture 5 tool loop with its own messages list; Finding is the Lecture 8 Grounded schema.

Five Rules for Orchestration

Each rule removes one family of failures from slide 20 — and each is enforced by the harness, not requested of the model.

1. **Start with one agent and tools.** Add an agent only for INFORMATION, PERMISSIONS, JUDGMENT or BREADTH — and beat a single agent on the same budget.
2. **Code owns control flow wherever you can write it.** The model decides the path only where you cannot.
3. **Every hand-off is a typed brief.** Objective, output schema, tools, boundaries, budget — a submittal, not a chat message.
4. **One writer per artifact.** Agents share files, not transcripts; pass paths, not copies.
5. **Checks and stops live in the harness.** A deterministic checker at each joint; max rounds, a token budget, and needs_human.

A rule the model reads is a suggestion; a rule the harness reads is a rule.

The Tools — Four You Already Have

A framework is a harness you didn't write (Lecture 6). Choose by where you want the control flow to live, and what you need to replay.

Tool	You define the team as	Control flow lives in	Use it when
Claude Code subagents	Markdown files in .claude/agents/ — own prompt, tools, model	the main agent, which delegates; a subagent returns only its result	capstone default: the checker on slide 25
Claude Agent SDK	Python: the Claude Code harness as a library, with subagents	the lead agent, inside your script	your agent becomes a program or a CI job
Raw Anthropic SDK	your own loop (slide 21)	your code	small teams you must audit line by line
LangGraph	StateGraph: nodes, edges, a typed shared State, a checkpointer	an explicit graph; supervisor or hand-off edges	you need replay, resume and human gates (Lecture 6)

Docs accessed October 2026: code.claude.com (subagents; agent teams for parallel sessions that message each other) · Claude Agent SDK · LangGraph multi-agent.

The Tools You Will Meet in Papers — and Two Protocols

Most multi-agent science papers since 2023 are built on one of these. Read them for their control flow, and translate.

AutoGen → Microsoft Agent Framework

Agents converse in a group chat; a manager picks the next speaker. Its successor adds typed graph workflows; AutoGen itself is in maintenance.

control flow: a chat manager

CrewAI

A crew of agents with role, goal and backstory; tasks run in a sequential or hierarchical process; Flows add explicit steps.

control flow: the process type

neuro-san (Cognizant AI Lab)

Agent networks declared in HOCON config files; agents route requests among themselves; private data travels outside the prompt (sly_data).

control flow: the network file

MCP · AGENT ↔ TOOL

Lecture 5. How a model finds and calls a tool: N × M integrations become N+M.

A2A · AGENT ↔ AGENT

How agents built by different vendors discover each other and exchange tasks. Both now sit under the Linux Foundation.

All three are model-agnostic; in this course run them with the pinned course model. Wu et al. 2023 (AutoGen); learn.microsoft.com/agent-framework; github.com/cognizant-ai-lab/neuro-san.

Register It — Your Agent's First Subagent

The Lecture 6 Checker, as a file in your capstone repo. Then the same split as a graph, for comparison.

`.claude/agents/checker.md`

```
---
name: checker
description: Verifies a Designer hand-off.
  Use after every design proposal.
  Never edits files.
tools: Read, Bash
model: inherit
---
You are the Checker. Run beam_tools.py
check on the proposed section. Return
PASS / FAIL / ABSTAIN with the tool output.
No tool output means ABSTAIN.
```

The same split in LangGraph (Lecture 6)

```
g = StateGraph(BeamState)
g.add_node("designer", designer)    # LLM
g.add_node("checker", run_checker)  # script
g.add_edge(START, "designer")
g.add_edge("designer", "checker")
g.add_conditional_edges("checker", route,
    {"revise": "designer",
     "done": END, "needs_human": END})
```

Information split: the checker sees the brief and the design, not the Designer's reasoning. Permission split: Read + the beam_tools command only — enforced in settings.json.

Subagents run in their own context window and return only their result. Optional this week; in the proposal, worksheet box 6 points at this file.

How Do You Know the Team Is Right?

A trace is evidence, not proof. A team leaves several traces — read the hand-offs first.

SAME-BUDGET BASELINE

One agent, same tools, same token budget, same held-out questions. The team must beat it (Discussion Point 1).

CHECKS AT EVERY JOINT

Count PASS / FAIL / ABSTAIN at each hand-off, not only at the end.

ONE TRACE PER AGENT

Tag each line with the agent that wrote it; 20 runs; label failures with MAST codes.

THE needs_human RATE

How often the team stops and asks. Zero is as suspicious as always.

Returns in Lecture 16 (evaluation, reliability, deployment) and the Week 9 audit of your persistent agent.

Lecture 9 — Takeaways, Readings, and Sunday

What is known Teams pay for breadth and separate contexts, cost ≈ 15× a chat, and fail at the hand-offs.

One or many Split only for information, permissions, judgment or breadth — and beat one agent on the same budget.

Design Answer control flow, shared state and hand-off on purpose: typed briefs, one writer, checks and stops in the harness.

Tools Pick a framework by where control flow lives. Capstone default: a Claude Code subagent.

Discussion Points: 1 same-budget baseline · 2 your box 6 · 3 the checker that never checked

READINGS

Required Schluntz & Zhang 2024 (re-read) · Hadfield et al. 2025 · Cemri et al. 2025 § § 1–4

Recommended Kim et al. 2025 · Yan 2025 · Swanson et al. 2025

Optional Wu et al. 2023 · Smit et al. 2024 · Elrefaie et al. 2025 · Minsky 1986

DUE SUNDAY 1 NOVEMBER, 11:59 PM

Lab Report 2: follow the TAs' guidelines. Weekly Guide & Check.

References — Lecture 9

Smith, R. G. (1980). The Contract Net Protocol. IEEE Trans. Computers C-29(12), 1104–1113.

Erman, L. D., Hayes-Roth, F., Lesser, V. R., Reddy, D. R. (1980). The Hearsay-II speech-understanding system. ACM Computing Surveys 12(2), 213–253.

Minsky, M. (1986). The Society of Mind. Simon & Schuster. · Wooldridge, M., Jennings, N. R. (1995). Intelligent agents: theory and practice. Knowledge Eng. Review 10(2).

Li, G. et al. (2023). CAMEL. NeurIPS. · Wu, Q. et al. (2023). AutoGen. arXiv:2308.08155. · Hong, S. et al. (2024). MetaGPT. ICLR. · Qian, C. et al. (2024). ChatDev. ACL.

Du, Y. et al. (2024). Improving factuality and reasoning through multiagent debate. ICML. · Smit, A. P. et al. (2024). Should we be going MAD? ICML, PMLR 235.

Ni, B., Buehler, M. J. (2024). MechAgents: Large language model multi-agent collaborations can solve mechanics problems, generate new data, and integrate knowledge. Extreme Mechanics Letters 67, 102131.

Schluntz, E., Zhang, B. (2024). Building Effective Agents. Anthropic [Ref_20]. · Hadfield, J. et al. (2025). How we built our multi-agent research system. Anthropic, 13 Jun 2025.

Yan, W. (2025). Don't build multi-agents. Cognition, Jun 2025. · Cemri, M. et al. (2025). Why do multi-agent LLM systems fail? NeurIPS; arXiv:2503.13657 [Ref_25].

Kim, Y. et al. (2025). Towards a science of scaling agent systems. arXiv:2512.08296 (v1 Dec 2025; v3 Apr 2026).

Swanson, K. et al. (2025). The Virtual Lab of AI agents designs new SARS-CoV-2 nanobodies. Nature. · Gottweis, J. et al. (2025). Towards an AI co-scientist. arXiv:2502.18864.

Ghafarollahi, A., Buehler, M. J. (2024). SciAgents. Advanced Materials, doi:10.1002/adma.202413523. · Elrefaie, M. et al. (2025). AI agents in engineering design. arXiv:2503.23315 [Ref_05].

Wallace, E. et al. (2024). The instruction hierarchy. arXiv:2404.13208.

In the news: Rath, P. et al. (2026). The thin orchestrator. Engineering at Meta, 19 Oct. · Sarkis, D. (2026). When to fold subagents into your main agent loop. Anthropic, 22 Oct. · Anthropic (2026). Dynamic Workflows, Claude Managed Agents docs (beta, 9 Oct). · Zhang, H. G. et al. (2026). The Virtual Biotech. bioRxiv 10.64898/2026.02.23.707551.

Documentation (accessed October 2026): Claude Code subagents · Claude Agent SDK · LangGraph · Microsoft Agent Framework · CrewAI · neuro-san · A2A · MCP.

Appendix A — The Fourteen MAST Failure Modes

Cemri et al. 2025. Use the codes when you label your own traces (slide 26).

1 · SPECIFICATION / SYSTEM DESIGN

1.1 Disobey task specification

1.2 Disobey role specification

1.3 Step repetition

1.4 Loss of conversation history

1.5 Unaware of termination conditions

2 · INTER-AGENT MISALIGNMENT

2.1 Conversation reset

2.2 Fail to ask for clarification

2.3 Task derailment

2.4 Information withholding

2.5 Ignored other agent's input

2.6 Reasoning–action mismatch

3 · TASK VERIFICATION

3.1 Premature termination

3.2 No or incomplete verification

3.3 Incorrect verification

Mode names as in the paper's taxonomy figure; the first family is called "specification issues" in v2 and "system design issues" in v3.